

4160 – Distributed and Parallel Computing

4160 – Distributed and Parallel Computing

High Performance Synchronization in Multi-core Era



CUHK

Multi-core CPU needs Atomic Instructions

- E.g., `test_and_set()` & `compare_and_swap()`
- E.g., Two `test_and_set(x)`s are concurrently executed by **different CPUs/cores** on the same memory location `x`
 - `x` will get updated by one by one
 - Can be used through newer versions of C/C++
 - E.g., C11

p.s. C11 has a Thread library, even easier to write multi-threading; that library is written using Pthread; programs using that are less portable than using Pthread directly

Mutual Exclusion – the complete picture (1)

- Lock-based

- Spin-based lock

- E.g., `pthread_spin_lock`

- What is inside?

- » Spin on 1 shared variable + **atomic instruction** to update it (problem: strict alternation)
 - » Spin on 2 shared variables + **atomic instruction** to update them (~Peterson; problem: priority inversion)
 - » Spin on shared variables + **atomic instruction** to update them in an even smarter way
 - (=Ticket, MCS algorithm, etc.)

- Sleep-based lock

- E.g., POSIX semaphore, `pthread_mutex_lock`

- What is inside?

- » `yield()` + **atomic instructions** to update the extra shared struct
 - » No waste the CPU to spin but let others use it (if any) 😊
 - » Generate context switch 😞

```
typedef struct {  
    int value;  
    list process_id;  
} semaphore;
```

Mutual Exclusion – the complete picture (2)

- Lock-free
 - No such a call like `pthread_lock_free`
 - Let the threads update a private copy of the shared data freely
 - Right before leaving the Critical Section
 - use atomic-instruction to test if there is any inconsistency
 - No? set the shared variable to the new updated value from the private copy
 - Yes? Race happened! Restart and Retry
- Others
 - More fine-grained locking // differentiate between writers and readers
 - Read Lock vs Write Lock // pthread has read-write lock implementation
 - if a writer gets a write-lock on item x, others' `acquire-read(x)` or `acquire-write(x)` have to wait
 - Mixing lock-based and lock-free // a famous one is called Optimistic Locking
 - Writers: acquire exclusive locks as usual
 - Readers: lock-free and re-try if conflict
 - // good for read-heavy workload because getting a lock (even that's a read-lock) requires writing (updating the lock status), which uses CAS and heavy

Basic Spinlock implementation using `test_and_set()`

Initialize `_Atomic lock = 0;`

```
while (TestAndSet(&lock) == 1) //spin_lock  
    ; //do nothing, spinning; this loop quits when 0==1
```

when reaching this point, =you get the lock → `lock = 1` *But others should be spinning*

```
//critical
```

```
.....  
//section
```

```
atomic lock = 0; //release the lock //spin_unlock
```

```
<remainder
```

```
section>
```

```
//pseudocode (indeed a single atomic instruction)  
boolean test_and_set(boolean *lock){  
    boolean initial = *lock; //original value  
    *lock = 1; //set  
    return initial; //return the original value!  
}
```

compare_and_swap()

```
//pseudocode (indeed a single atomic instruction)
boolean cas(int *value, int expected, int new) {
    if (*value != expected) {
        return false;
    }
    value = new;
    return true;
}
```

Basic Spinlock implementation using CAS

- lock – a shared variable

```
Initialize _Atomic lock = 0;
thread-function() {
while (!compare_and_swap(&lock, 0, 1))
    { ; }//
//critical
.....
//section
atomic lock = 0; //release the lock//unlock
<remainder
section>
}
```

This can be wrapped as 'spin-lock()'



```
//pseudocode (indeed a single atomic instruction)
boolean cas(int *value, int expected, int new) {
    if (*value != expected) {
        return false;
    }
    *value = new;
    return true;
}
```

Check your knowledge

- Do you see pthread_mutex_lock?
- Is it lock-free?

```
Initialize _Atomic lock = 0;
thread-function() {
while (!compare_and_swap(&lock, 0, 1))
    { ; }// //spin lock
//critical
    .....
//section
atomic lock = 0; //release the lock
<remainder

section>
}
```

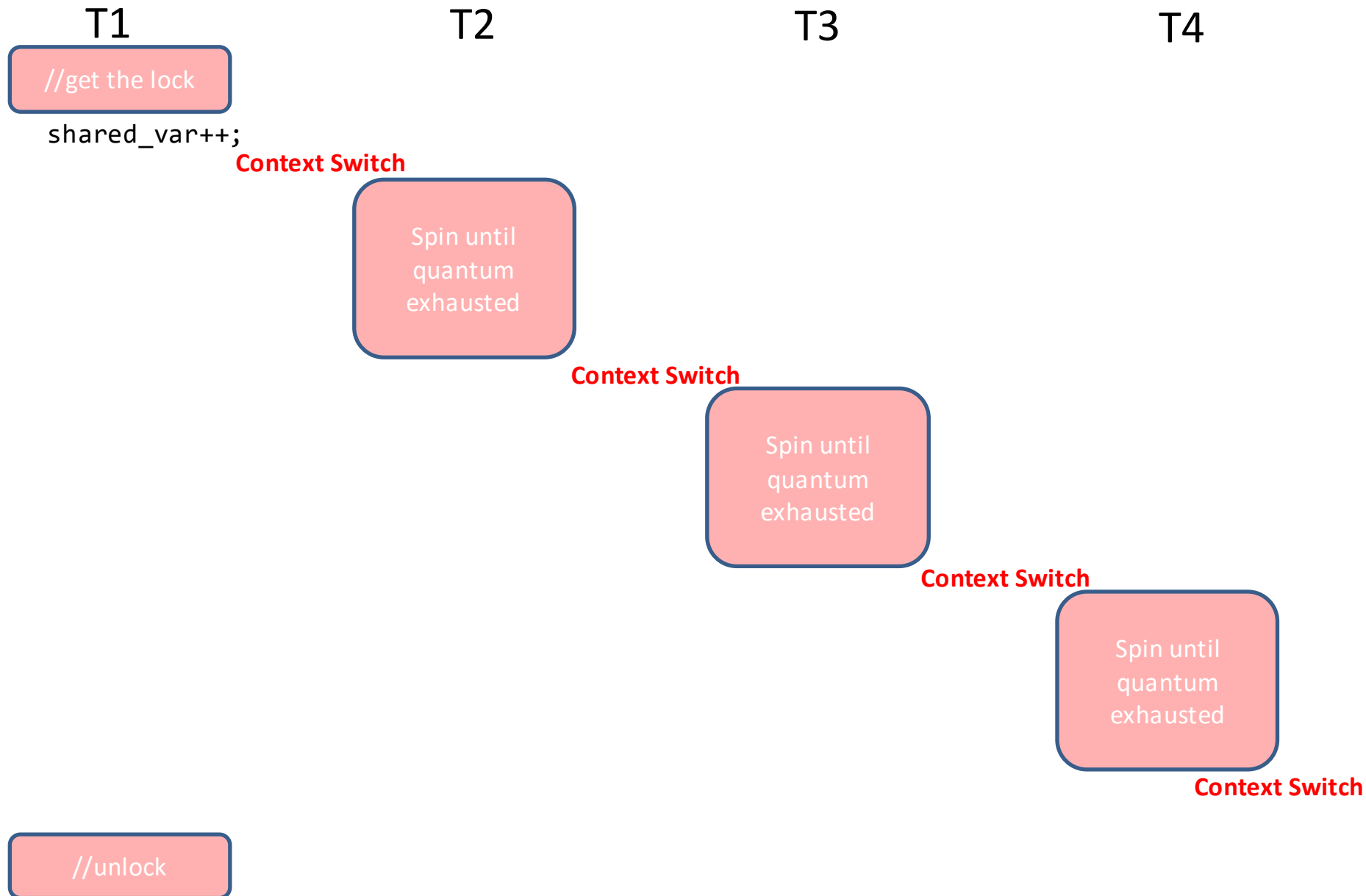

Counting: Lock-based vs. Lock-free

```
Initialize _Atomic lock = 0;
thread-function() {
    while (!compare_and_swap(&lock, 0, 1))
        { ; } // spin lock
    //critical
        shared_var++;
    //section
    atomic lock = 0; //release the lock
    <remainder
section>
}
```

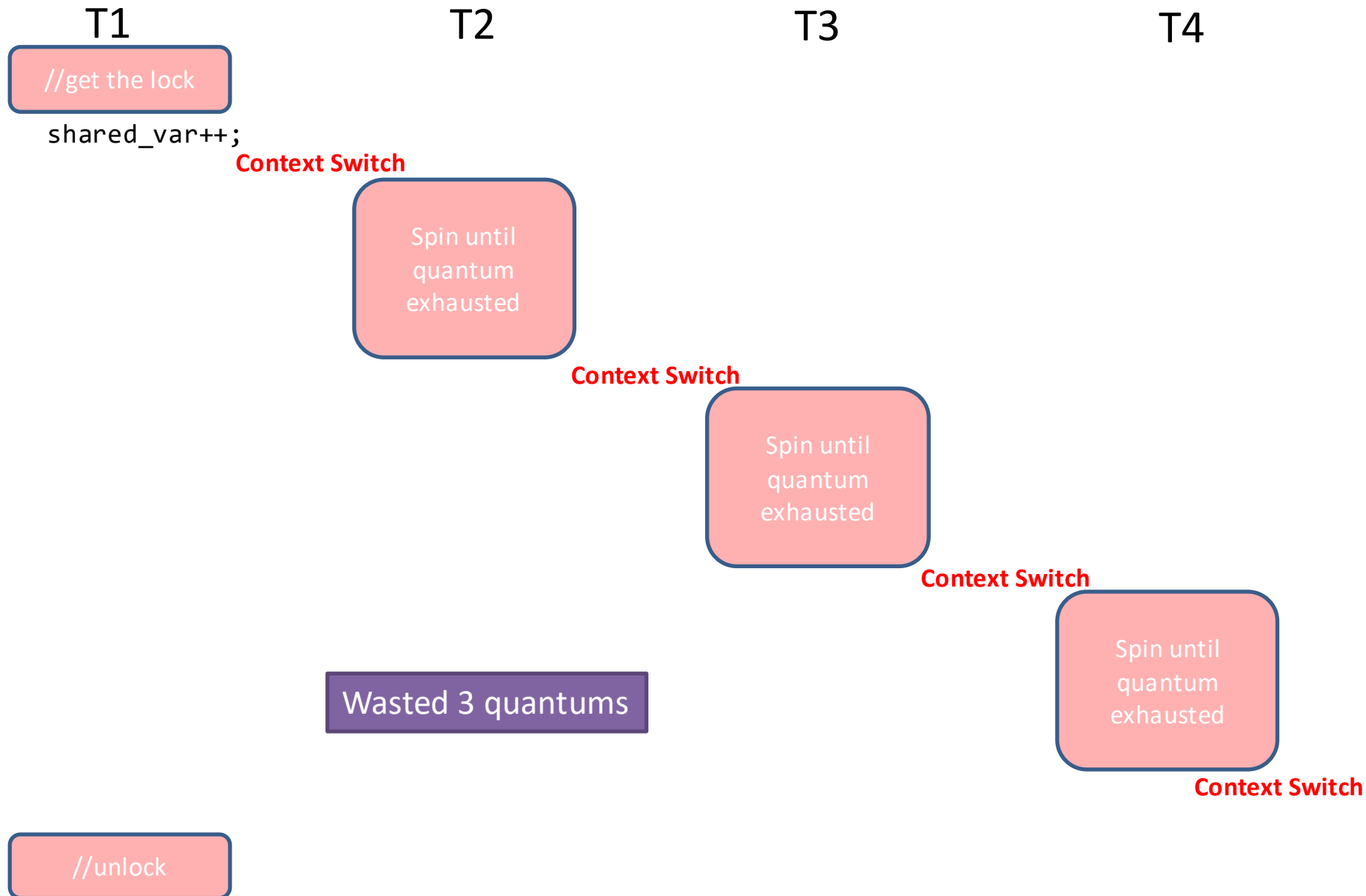
```
Initialize _Atomic lock = 0;
thread-function() {
    do {
        //critical
        1st_read = shared_var; //optimistic
        new = 1st_read++; //do the CS first
        //section //but do it thread-locally
    while (!cas(&shared_var, 1st_read, new))
        <remainder
        //redo if contaminated
        //else install
    section>
}
```

```
boolean cas(int *value, int expected, int new) {
    if (*value != expected) {
        return false;
    }
    value = new;
    return true;
}
```

Counting: SpinLock-based (4 threads; 1 core)



Counting: SpinLock-based (4 threads; 1 core)



Counting: Lock-free (4 threads; 1 core)

T1

T2

T3

T4

1st_read=10

Context Switch

1st_read=10

Context Switch

1st_read=10

Context Switch

1st_read=10

Context Switch

cas(10, 10, 1st_read++);
(success!)

cas(11, 10, 1st_read++);
Contaminated! cas failed

RE-DO! success!

Wasted 1 quantum

cas fail

RE-DO!

Wasted 1 quantum

cas fail

RE-DO!

Wasted 1 quantum

Lock-free summary

- Writer-Reader conflict:
 - Writer:
 - Make every change **thread-private/local**
 - One atomic CAS to validate and swap the final (safe) result and make things public
 - No in-flight intermediate result is exposed to other threads
 - Reader:
 - Since they never observe intermediate changes from writers
 - Always success w/o retry
- Writer-Writer conflict:
 - Conflicting writers:
 - One success CAS
 - Others abort and restart

Lock-free needs to take care of the ABA problem

Time

Thread 1:
Pop(*lfstack){
 //Originally, lfstack->head=0x01
 orig = atomic_load(lfstack); //orig.head=0x01
 next.head = orig.head->next; //next.head=0x02

Context switch

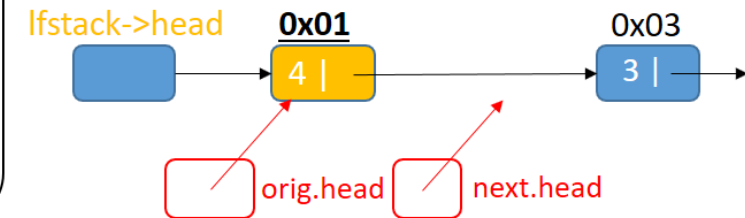
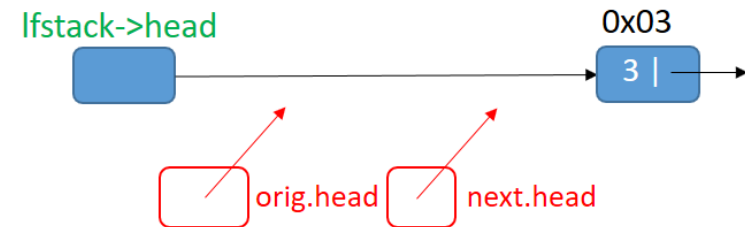
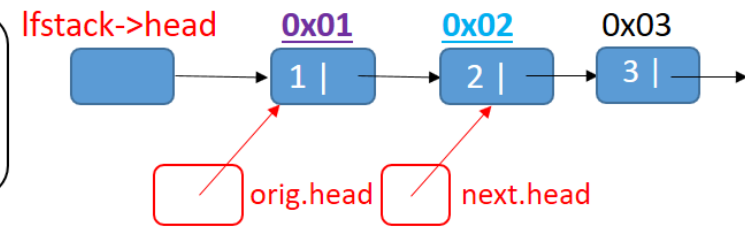
Thread 2:
Pop(*lfstack) //free 0x01
Pop(*lfstack) //free 0x02
 //now, lfstack->head=0x03

Context switch

Thread 3:
Push(*lfstack, 4){
 Node *node = malloc(sizeof(Node)); //allocator reuses the same memory
 // block 0x01, so node=0x01
 ...
 //when thread 3 pushes **node** into the stack, it let
 //lfstack->head point to **node**, then, lfstack->head=0x01;
}

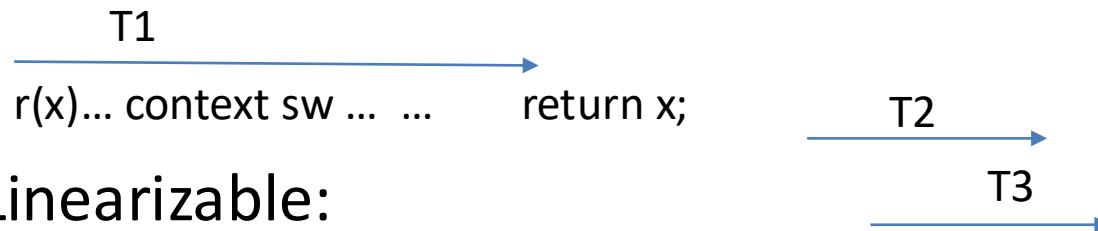
Context switch

Thread 1:
 CAS(lfstack, &orig, next); //here, it compares the content points to by
 //lfstack with the content of orig, that is, lfstack->head and orig.head, for equality.
 //For lfstack->head, which is 0x01, while for orig.head, which is also 0x01, they are
 //the same, then CAS succeeds! So it sets lfstack->head=next.head=0x02! Then,
 //lfstack->head will point to a freed memory! Error!
}



The correctness of shared data structure

- Unlike distributed, accept only one level: Linearizability
 - Behave like a single-thread on a single-core
 - Hence = linear order that **respects real-time**



- Linearizable:

- All threads see the same **linear order** that **respects real-time**
- **T1** T2 T3 or **T1** T3 T2
 - With context switch, nobody can ensure T2 vs T3

- Serializable:

- All threads (transactions) see the same **serial** order
 - Not necessary respects real-time
- Easier: T1 T2 T3, T1 T3 T2, T2 T3 T1, T2 T1 T3, T3 T1 T2, T3 T2 T1
- Harder: Isolation across transactions (multiple items)

Linearizability

- Can you find a linear order from the history (that respects real-time) here?

T1	<u>a) Wx0</u>	<u>b) Wx1</u>
T2		<u>c) Wx2</u>
T3	<u>d) Rx2</u>	<u>e) Rx1</u>
T4	<u>f) Rx1</u>	<u>g) Rx2</u>

- $c \rightarrow d$ //otherwise T3's first read can't see $x=2$
- $d \rightarrow b$ //otherwise T3's first read would see $x=1$
- $b \rightarrow f$ //otherwise T4's first read can't see $=1$
- $f \rightarrow c$ //otherwise T4's first read see $x=2$
- Cycle!

Cycle → Not Linearizable
No Cycle not necessarily implies
Linearizable

One solution to ABA problem

- Add a 'counter' to indicate how many people have modified it before
- So, even if the content is the same, we can still detect inconsistency

Lock-free vs Lock-based



- Lock-based incurs problems of:
 - **Lock holder preemption**
 - A thread gets a lock, but gets context switch away, others wait for locks → **hurt performance**
 - **Deadlock**
 - No system-wise progress during deadlock
 - Potential In-scalability
 - **Primitive mutex implementation incurs 2 shared memory writes** (lock/unlock) per CS access
- Lock-free is free of the above but
 - Very difficult to code right (e.g., ABA)
 - More complex code → more instructions
 - In case of high contention, restart many times
 - CAS is expensive

- On OS
 - # of processes >> # of cores
 - Lock-free is great → a dead/slow process/thread **accessing a shared data structure in the kernel** won't **halt the system** (e.g., imagine a thread dies with a lock...)
 - OS data structures are simple (e.g., stack, linked-list)
 - Easier to code right, not much overhead
- On high-performance systems (e.g., DB, kvstore)
 - CAS is expensive (because of sync)
 - Complex structures (e.g., B+-tree)
 - have way more complicated lock-free logic (e.g., ABA)
 - advanced locking schemes (e.g., optimistic locking, fine-grained rw locking) might have better performance
 - But lock-free has one definitive advantage
 - -> **readers never get blocked/retry**

Lock-free: readers never get blocked/retry

```
get() { //reader
    cout << shared_var;
}
```

//when cas installs, it is final and clean

Initialize _Atomic lock = 0;

```
put() {
    do {
        //critical
        1st_read = shared_var;
        new = 1st_read++;
        //section
        while (!cas(&shared_var, 1st_read, new))
            <remainder
    }
    section>
}
```

//optimistic
//updates are
thread-private

//redo if contaminated

```
boolean cas(int *value, int expected, int new) {
    if (*value != expected) {
        return false;
    }
    value = new;
    return true;
}
```

Lock-free vs Wait-free: non-blocking algorithm

Lock-free	Wait-free
• Guarantee progress for some threads, • Regardless the state/speed of other threads • Guarantee system-wise progress • it rules out the lock-holder-preemption problem • Still, a particular thread can suffer from starvation • Thread A keeps on retrying while a late-comer B can be served first	• Guarantee progress for every thread, • Regardless the state/speed of other threads • Guarantee per-thread progress • Sometimes impossible to achieve • Thread1.opX() must finish in a finite number of steps • Regardless of the state/speed of the other thread • I.e., starvation-free
E.g., lock-free stack, lock-free hashtable	E.g., The lockless page cache patches to the Linux kernel are an example of a wait-free component.

Which one is easier to achieve?

Connecting Parallel and Distributed

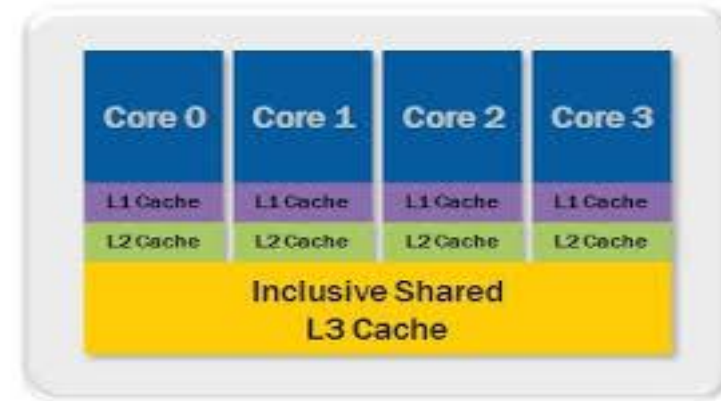
- Wait-free can be viewed as a 'liveness' property
- Synchronization can be viewed as 'consensus'
- M. Herlihy thus reuses FLP-proving techniques to prove that:
 - there is no wait-free multi-thread (>1) algo that can do synchronization if the access interface is *read()* and *write()* only
 - if the given interface is *cas()* (regarded as a member of *rmw()* family), then wait-free algo is possible.

CAS in C++11 (programming)

- C++11's CAS is not restricted to `int` type only.
 - The more usual one is **pointer-based**.
- C++11's CAS has weak and strong version
 - `_Bool atomic_compare_exchange_weak(volatile A *object, C *expected, C desired)`
 - Google the meaning of
 - Volatile
 - A: atomic
 - C: non-atomic
 - For our course using x86, no difference

Memory Consistency Model

- Weak and strong refer to the memory consistency model
 - At the beginning, Core 1:
 - LOAD memory address 0x888888 (content="old")
 - STORE *0x888888 = "new"
 - Update stills in Core 1's local L1
 - (not yet flush to memory)
 - Then, Core 2:
 - Read memory address 0x888888
 - Whether core 2 reads "old" or "new"
 - Strong consistency memory model
 - x86 (i.e., Intel and AMD) → **cache coherence** → core 2 reads "new"
 - Weak consistency memory model
 - ARM (Advanced RISC Machine) → almost all mobile devices → no cache coherence → core 2 reads "old"
 - To avoid that happens, programmers have to do "defensive programming"
 - » explicit add **memory fence** themselves



Spinlock Implementation

- Lock-based
 - Spinlock
 - Basic Spinlock using 1 shared variable is one implementation
 - easily violate progress
 - Peterson spinning using 2 shared variables
 - Suffer from priority inversion
 - **Ticket Spinlock is a better implementation**
 - Using another atomic instruction
 - **MCS Spinlock is an efficient implementation**
 - Using another atomic instruction + CAS
 - Good when there are lots of waiting threads

What's bad about this Spinlock implementation?

```
while (TestAndSet(&lock) == 1)
    ; //do nothing, spinning; this loop quits when 0==1

//critical
.....
                                //section
atomic lock = 0; //release the lock
<remainder

section>
```

What's bad about this Spinlock implementation?

```
while (TestAndSet(&lock) == 1)
    ; //do nothing, spinning; this loop quits when 0==1

//critical
.....
                                //section
atomic lock = 0; //release the lock
<remainder

section>
```

Background:

All atomic instructions are expensive
(because of cache coherence, locking
the memory bus, etc.)

Bad 1: how many times TAS is used here?
=WQ

where W=# of waiting threads and
Q=num of loops in a quantum

Bad 2: no guarantee FIFO among the
threads who are competing for the lock

What about this ticket-based Spinlock implementation?

```
ticket_lock{  
    int now_serving;  
    int next_ticket;  
};
```



```
function Lock(ticket_lock lock){  
    //get your ticket atomically  
    int my_ticket = fetch_and_increment(lock.next_ticket);  
    while(my_ticket != now_serving){//spin read}  
}
```

```
function Unlock(ticket_lock lock){  
    lock.atomic(now_serving++);  
}
```

Intuition:

- Spin **read** on a **shared variable** instead of spin with an **atomic instruction**
- Get a ticket and spin on the ticket

fetch_and_increment(x)

Atomic instruction:

- Return x;
- But x++;

What about this ticket-based Spinlock implementation?

```
ticket_lock{
    int now_serving;
    int next_ticket;
};

function Lock(ticket_lock lock){
    //get your ticket atomically
    int my_ticket = fetch_and_increment(lock.next_ticket);
    while(my_ticket != now_serving){//spin read}
}

function UnLock(ticket_lock lock){
    lock.atomic(now_serving++);
}
```

- 1) Every thread gets a unique ticket #
- 2) How many f_a_i each thread used?
- 3) Threads get the lock in FIFO order

fetch_and_increment(x)
Atomic instruction:

- Return x;
- But x++;

Is ticket-based Spinlock perfect?

(Assume 4 cores with C1 gets the lock)

```
ticket_lock{
    int now_serving;
    int next_ticket;
};

function Lock(ticket_lock lock){
    //get your ticket atomically
    int my_ticket = fetch_and_increment(lock.next_ticket);
    while(my_ticket != now_serving){//spin}
}

function Unlock(ticket_lock lock){
    lock.atomic(now_serving++);
}
```

1) Every core except the lock holder, core C1, now spins on now_serving

So, what?

Recall about cache coherence

2) Cores 2-4 are spinning:

- Read now_serving
- now_serving in each's L1 cache

3) Now, assume C1 releases the lock
→ now_serving++

4) x86 cache coherence will invalidate all other local copies of now_serving and ask them to go C1 to get the latest now_serving value

Is ticket-based Spinlock perfect?

```
ticket_lock{
    int now_serving;
    int next;
};

function Lock(
    //get your ticket
    int my_ticket;
    while(my_ticket != now_serving)
}

function UnLock(
    lock.atomics
}
```

1) Every core except the lock holder, say C1, now spins on

On that unlock, 3 copies of now_serving has to move from C1 to {C2, C3, C4}

i.e., the unlock cost is $O(W)$, where W is # of cores waiting for the lock

Any method to further reduce that down to $O(1)$?

Yes, MCS proposed one such implementation

Note: ticket-based: now_serving is the culprit because it is shared among all cores

4) x86 cache coherence will invalidate all other local copies of now_serving and ask them to go C1 to get the latest now_serving value

MCS Intuition

```
mcs_node{
    mcs_node next;
    int spin;
}

mcs_lock{
    mcs_node queue;
}
```

- When you lock
 - if someone else holds the lock, then add yourselves to the end of the **queue**. You then busy wait on **your own local spin** field.
- When you unlock
 - if there is a core waiting on you, we need to set the **spin** of your successor to false
 - like passing the token to the next

MCS Lock

- Atomically place my node at the end of the queue.
- Return the original tail

```
function Lock(mcs_lock lock, mcs_node my_node){
    my_node.next = NULL;
    mcs_node predecessor = fetch_and_append(lock.queue, my_node);
    //if it's null, nobody is holding the lock, so Lock() returns → i.e., enter CS!
    if (predecessor != NULL){//else someone is waiting already, +myself to the queue
        my_node.spin = true;
        predecessor.next = my_node;
        while(my_node.spin){} //spin until my predecessor gives me the lock
    }
}
```

```
mcs_node{
    mcs_node next;
    int spin;
}

mcs_lock{
    mcs_node queue;
}
```

MCS Unlock

```
function Lock(mcs_lock lock, mcs_node my_node){
    my_node.next = NULL;
    mcs_node predecessor = fetch_and_append(lock.queue, m
//if it's null, nobody is holding the lock, so Lock()
    if (predecessor != NULL){//else someone is waiting al
        my_node.spin = true;
        predecessor.next = my_node;
        while(my_node.spin){} //spin until my predecessor
    }
}
```

1) It is possible that a core has just **f_and_a** but not yet **run this line;** e.g., context switched

```
function Unlock(mcs_lock lock, mcs_node my_node){
    //is there anyone follow me so that I can give the lock to?
    if (my_node.next == NULL){
        //double check!
        if (compare_and_swap(lock.queue, my_node, NULL)){
            return;
        }
        else{
            //someone x has executed f_a_s but not set my next field as x
            while(my_node.next==NULL){//spin}
        }//end else
    }
    //if we made it here, there is someone waiting, so lets wake it up
    my_node.next.spin=false;
}
```

2) So, **CAS** to ensure the queue tail is genuinely empty

If yes, then I am really the last one in the queue, flag nobody and unlock finishes

MCS Implementation

```
function Lock(mcs_lock lock, mcs_node my_node){
    my_node.next = NULL;
    mcs_node predecessor = fetch_and_append(lock.queue, my_node);
    //if it's null, nobody is holding the lock, so Lock() returns→i.e., enter CS!
    if (predecessor != NULL){
        my_node.spin = true;
        predecessor.next = my_node;
        while(my_node.spin){} //spin
    }
}

function Unlock(mcs_lock lock, mcs_node my_node){
    //is there anyone follow me so that I can give the lock to?
    if (my_node.next == NULL){
        //need to make sure there wasn't a race here
        if (compare_and_swap(lock.queue, my_node, NULL)){
            return;
        }
        else{
            //someone has executed f a s but not set my next field
            1) while(my_node.next==NULL){} //spin
        }
        //if we made it here, there is someone waiting
        my_node.next.spin=false;
    }
}
```

2) Spin until that someone else finishes its job - set her as my_node.next

MCS lock, how good is it?

```
function Lock(mcs_lock lock, mcs_node my_node){
    my_node.next = NULL;
    mcs_node predecessor = fetch_and_append(lock.queue, my_node);
    //if it's null, nobody is holding the lock, so Lock() returns→i.e.. enter CS!
    if (predecessor != NULL){
        my_node.spin = true;
        predecessor.next = my_node;
        while(my_node.spin){} //spin
    }
}
```

```
function Unlock(mcs_lock lock, mcs_node my_node,
    //is there anyone follow me so that I can give the lock to?
    if (my_node.next == NULL){
        //need to make sure there wasn't a race
        if (compare_and_swap(lock.queue, my_node, NULL))
            return;
        else{
            //someone has executed f_a_s but
            while(my_node.next!=NULL){} //spin
        } //end else
    }
    //if we made it here, there is someone waiting
    my_node.next.spin=false;
}
```

of atomic instructions?

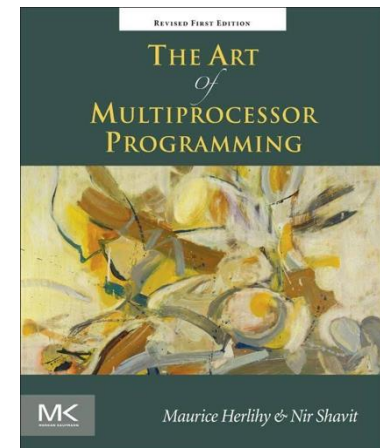
- Lock(): 1 atomic instruction (like Ticket-based)
- Unlock(): 1 atomic instruction

Ticket-based: now_serving was the culprit that is shared among all cores => cache-coherence cost on update

MCS: now everyone just spins on its **OWN LOCAL my_node.spin**
Unlocking just notifies one person but not all

MCS is a spin-lock?

- It is so famous that some gives it a standalone name
 - queue lock
- Further readings:
 - Scaling Synchronization in Multicore Programs
 - <https://queue.acm.org/detail.cfm?id=2991130>
 - The Art of Multiprocessor Programming



Lock-based: semaphore vs spin vs sleep

- Spin vs sleep:
 - Sleep-based (e.g., semaphore) was dominating in 1-core era
 - You get the precious only core and spin?! You better go to sleep
 - Nowadays, many-core and main-memory oriented
 - Sleep-based context switch is too expensive
 - Core is fast and most spins are short
 - So, spinlock isn't that bad
- Sleep-based: semaphore vs pthread_mutex_lock
 - Semaphore is general but harmful
 - General:
 - Global: every thread/process can update the semaphore
 - Used by both processes and threads
 - Can be used for synchronizing non-binary resources
 - Harmful:
 - Vs Pthread_mutex_unlock is local: only the thread which locked can unlock
 - » Someone else can unlock your lock!
 - Use Pthread lock and conditional variable if possible

Advanced lock type beyond mutex

- Reader-Writer lock (Shared-eXclusive) lock

- Finer grained -> higher concurrency

- But still hurt scalability

- read_lock() {

- set `shared_mem_read_lock_bit` = 1;}

	S	X
S	✓	X
X	X	X

Ref: [geeks4geeks](#)

- Optimistic Lock (State-of-the-art)

- Usage: the same as rw-lock //hence, also fine-grained

- But more efficient because (read) lock involves no shared_memory write //more scalable

Optimistic Lock (a.k.a version lock)

- 64-bit Lock = <version + lock-bit>
 - Put(k): //lock-based + version

```
Write-lock(n) { //this node contains k
    if lock-bit=1
        restart;
    set lock bit=1; //exclude other writers }
Add k to node n
Write-unlock(n) {
    set version++, set lock-bit =0} //put lock and version in the same word
```
 - Get(k): //similar to lock-free

```
restart: //but no guarantee of progress because of the restart loop here

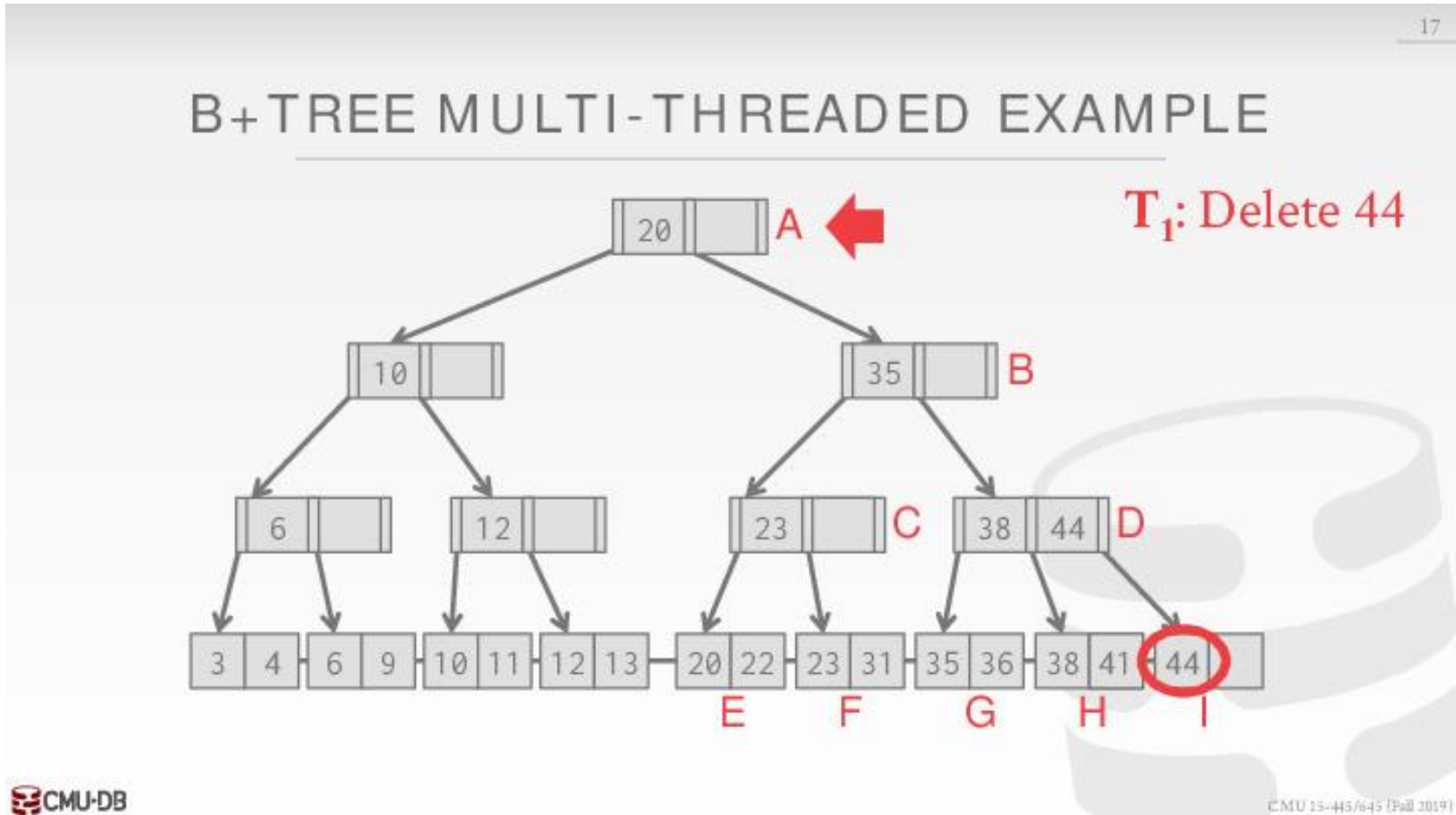
ver = Read-lock(n){
    If lock=1 → goto restart //No shared memory write here ☺
    Else return version}

Read k from node n

success = Read-unlock(ver) {
    If version unmatched or lock is on} //since we didn't block writers, hence you may read stale
If !success → goto restart
```
- Vs True lock-free: readers (i.e., get(k)) never restart

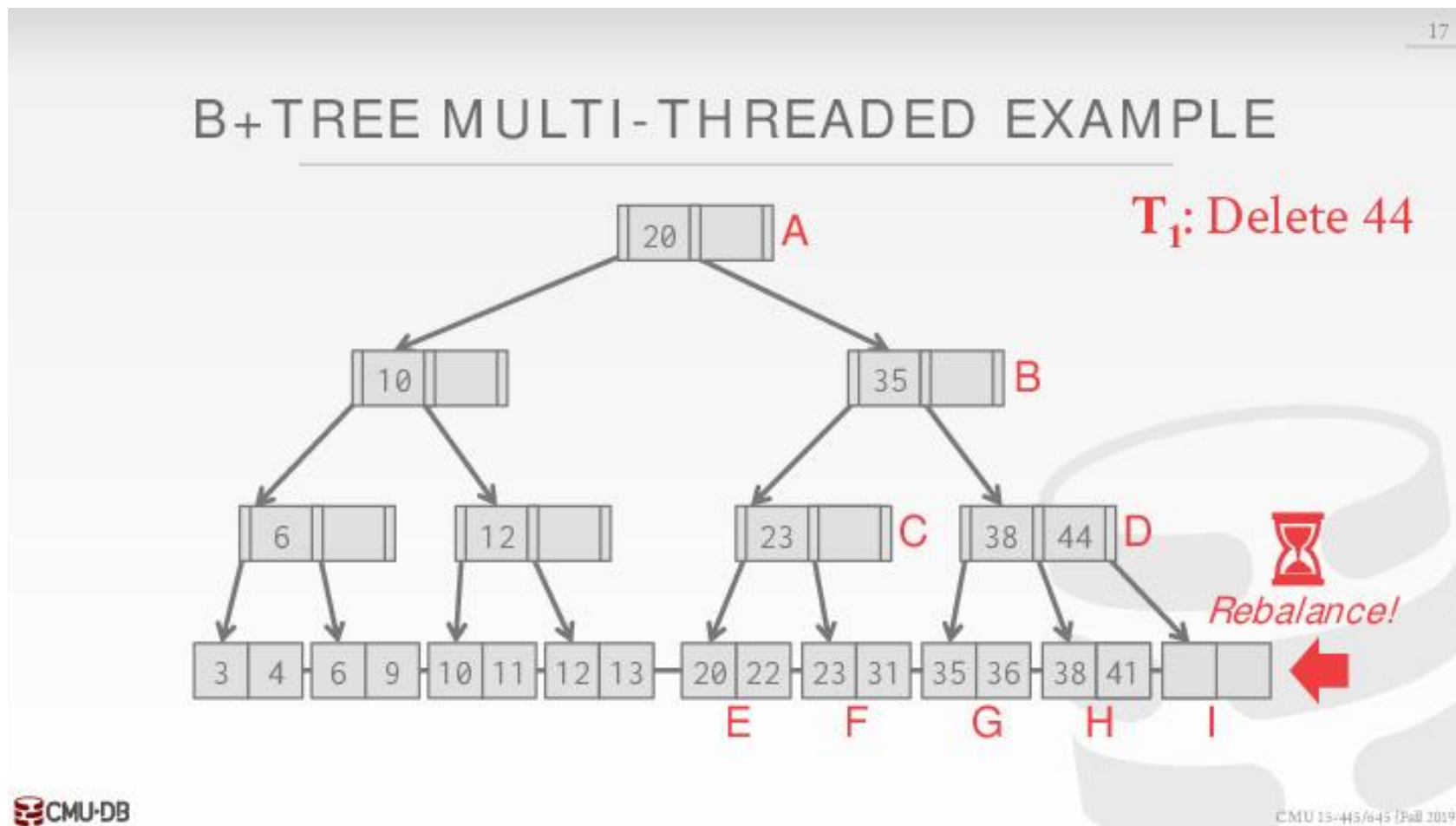
Concurrency in complex structures

- E.g., B+tree



Concurrency in complex structures

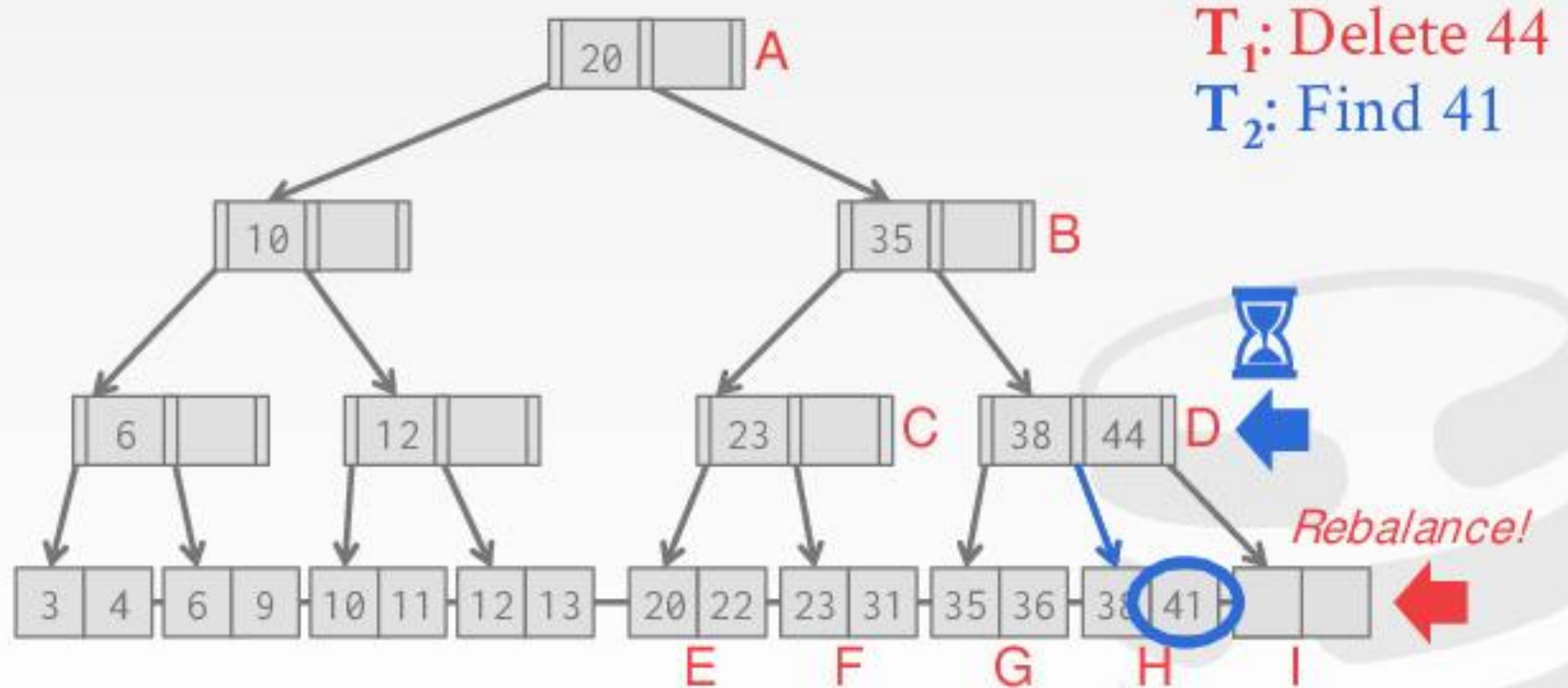
- E.g., Trigger SMO (rebalance) but context switch to T2



T2 arrives, reach D, just get the **ptr**, then cxt swit

17

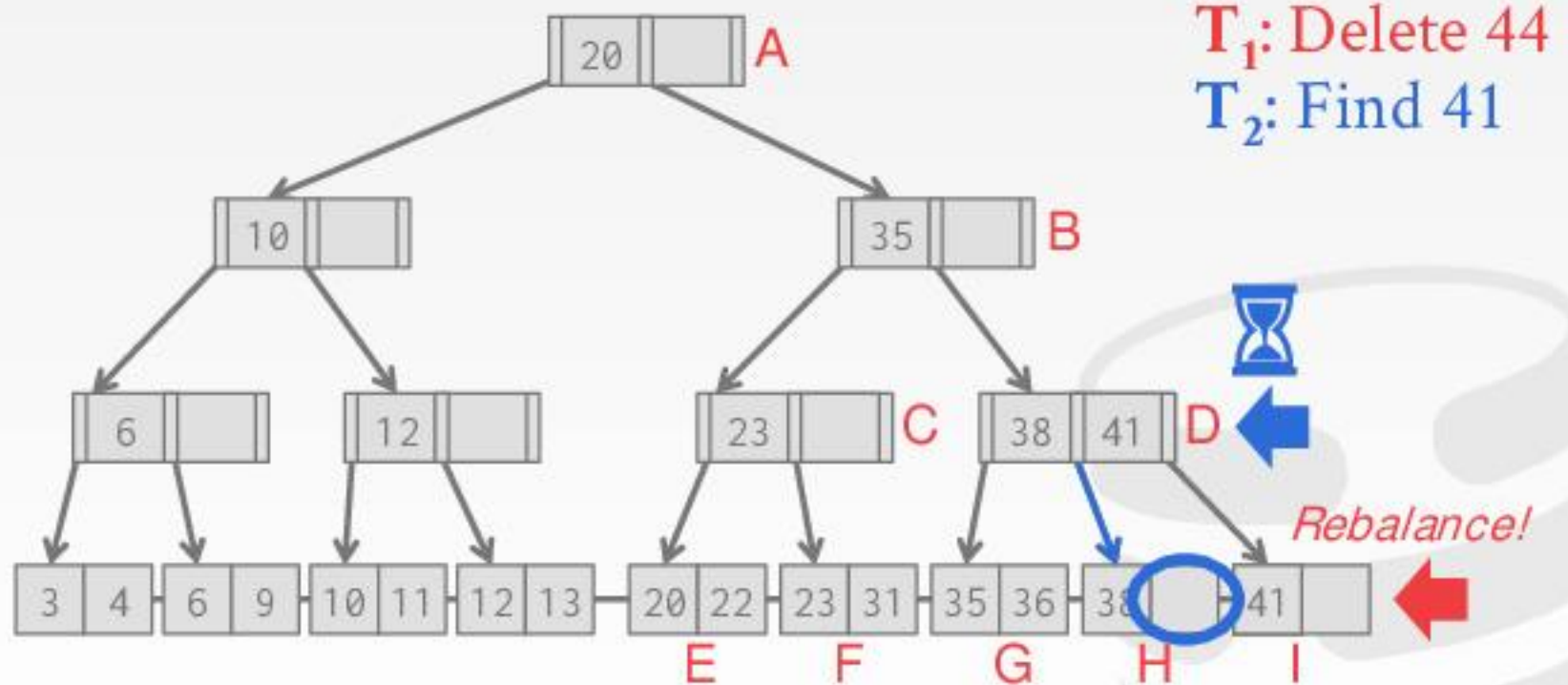
B+TREE MULTI-THREADED EXAMPLE



T1 continues its rebalancing

17

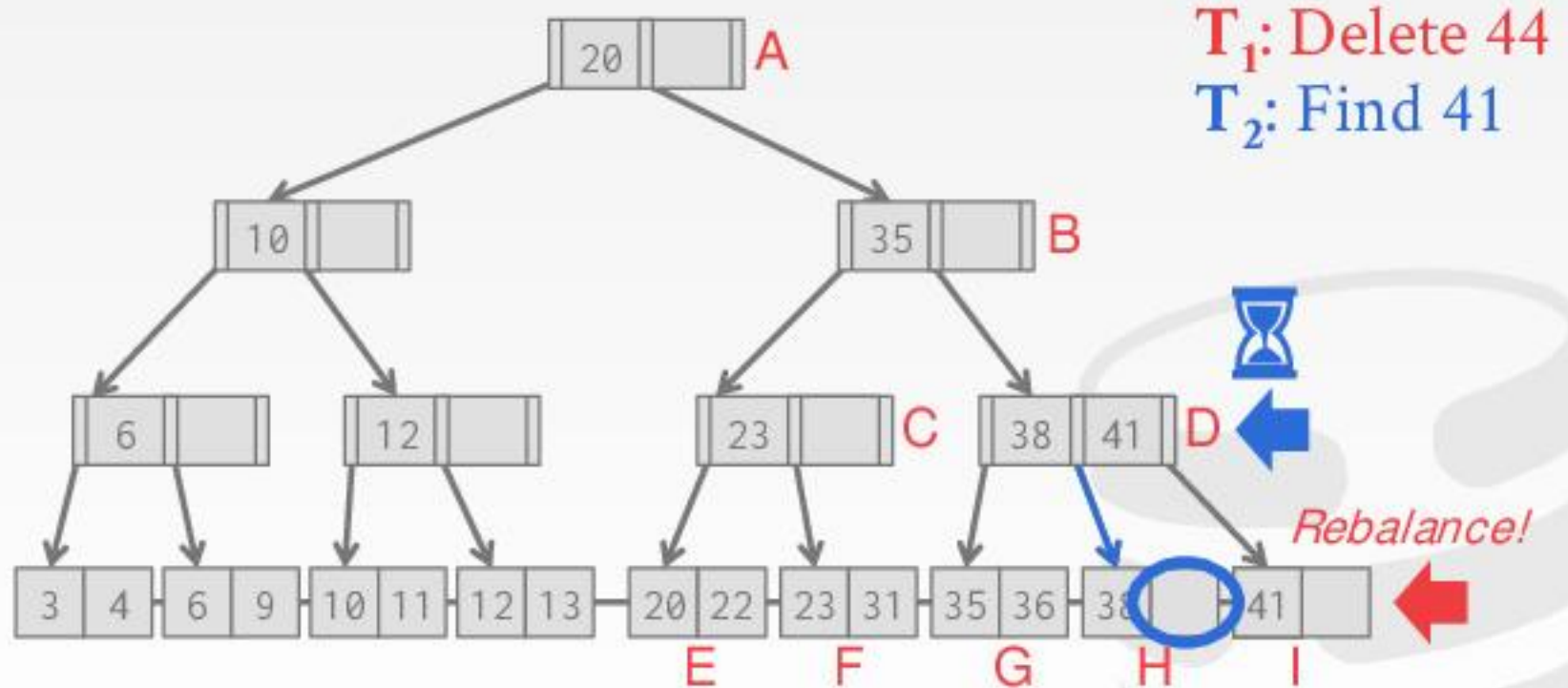
B+TREE MULTI-THREADED EXAMPLE



Context switch back to T2 → T2 said “no 41”

17

B+TREE MULTI-THREADED EXAMPLE



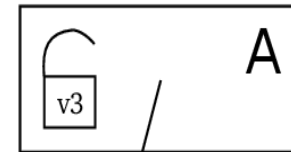
Solution: Lock coupling

- Find:
 - Acquire R-lock on child
 - Then unlock parent
- Insert/Delete:
 - Obtain W-lock if needed
 - Once a child is locked
 - Check if the child is safe (won't split or merge anymore)
 - If safe:
 - » can release locks on ancestors



traditional

1. lock node A
2. access node A



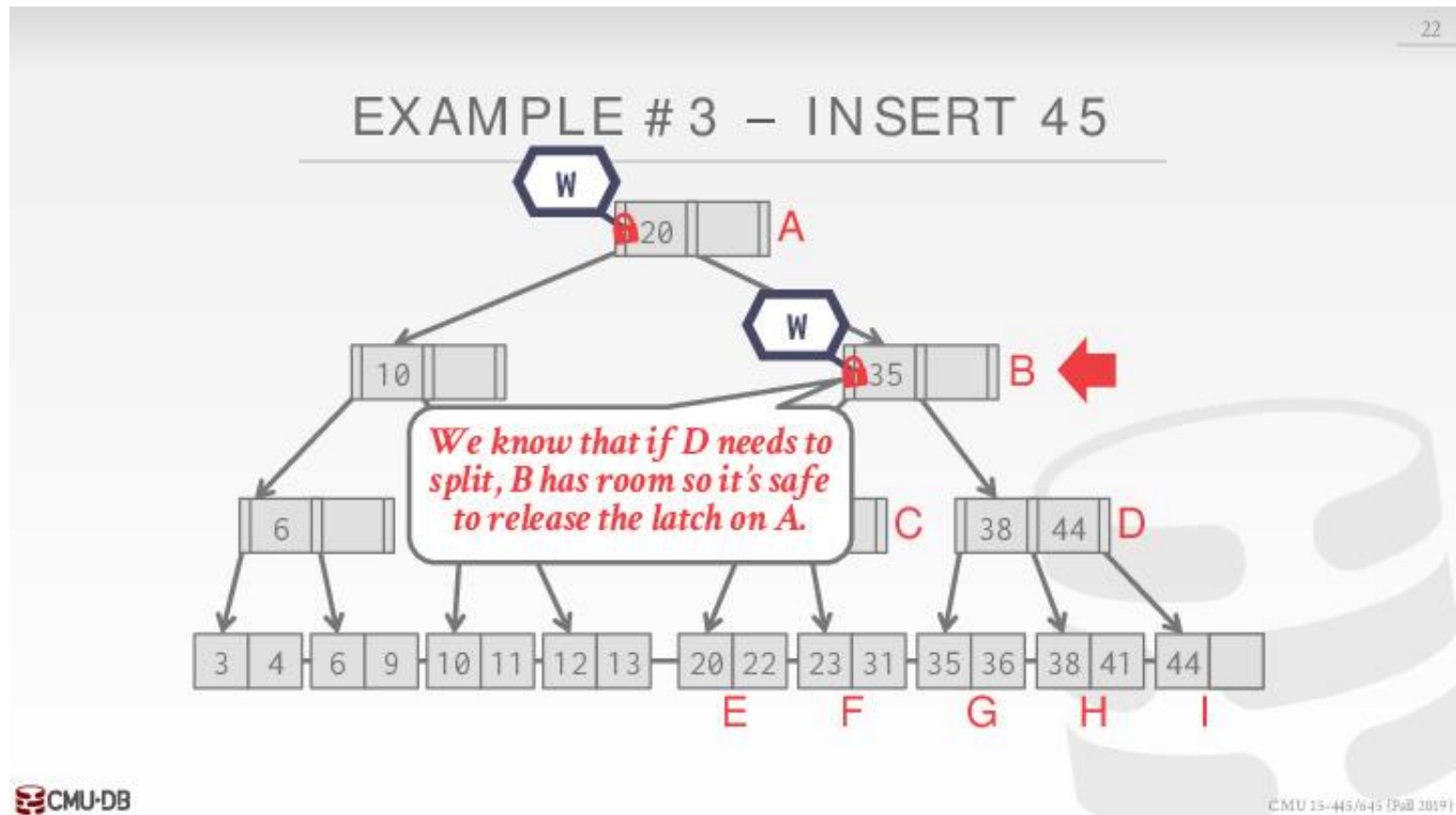
3. lock node B
4. unlock node A
5. access node B



6. lock node C
7. unlock node B
8. access node C
9. unlock node C



One case, at B



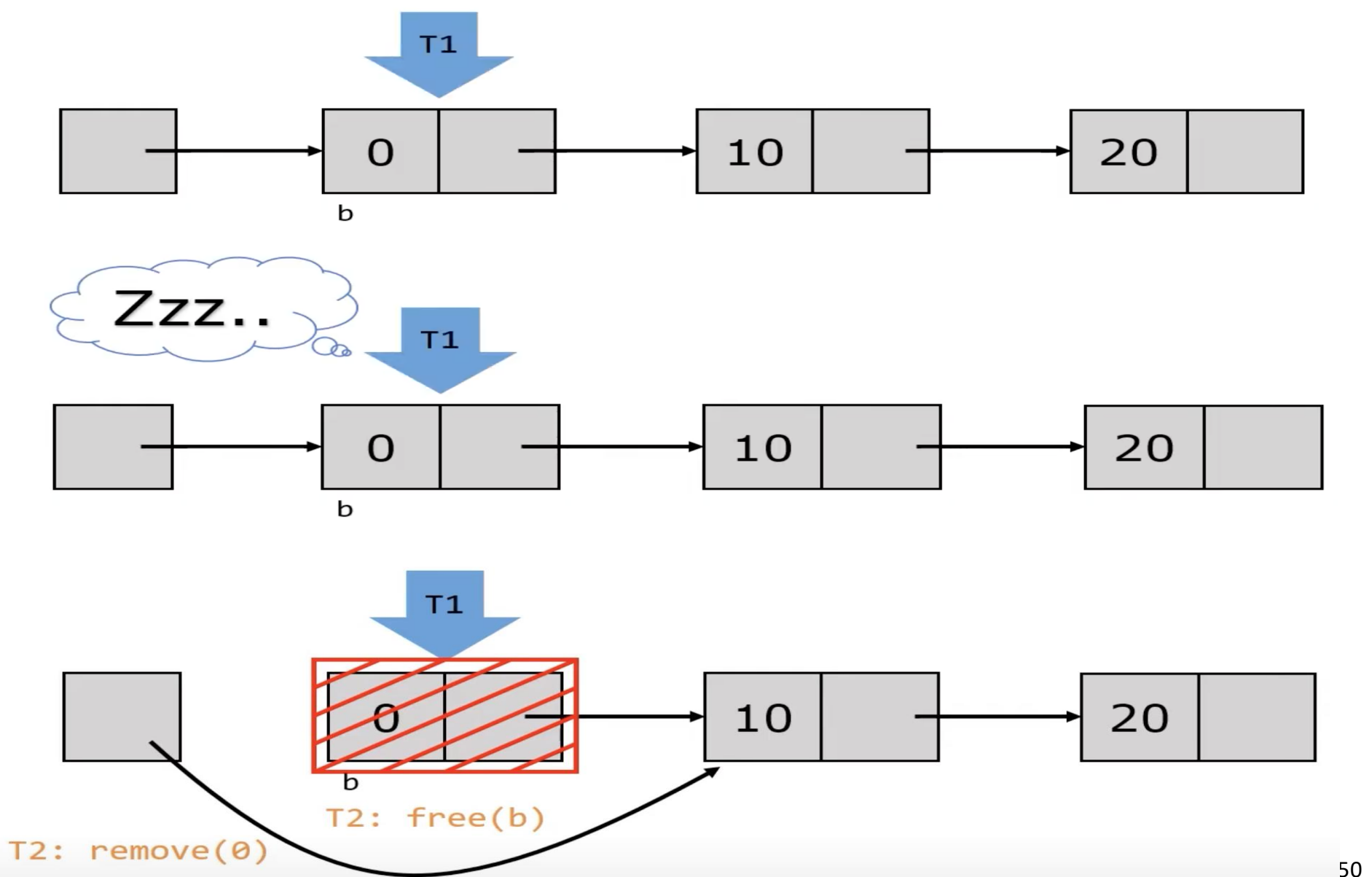
23

CMU 15-445/645 (Fall 2019)

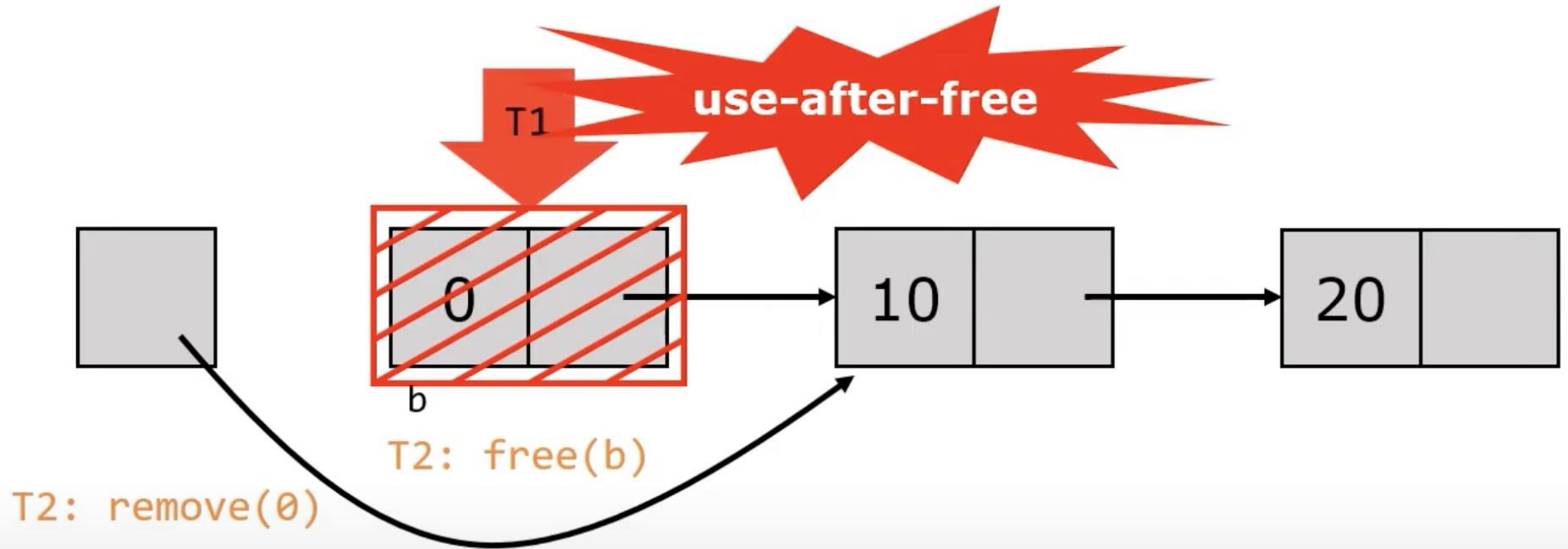
Memory reclamation

- It is trivial in lock-based world
- If you are deleting a key (and freeing a node n) in a data structure
 - You are still holding the lock
 - You won't release the lock until you finish delete and free the node
 - Others won't be able use-after-free since they are waiting for the lock
- In lock-free world
 - (including optimistic locking because its reader essentially 'lock-free', though it would restart, not 100% pure lock-free)
 - the timing that you can free a node becomes tricky

General memory reclamation in lock-free structures



General memory reclamation in lock-free structures

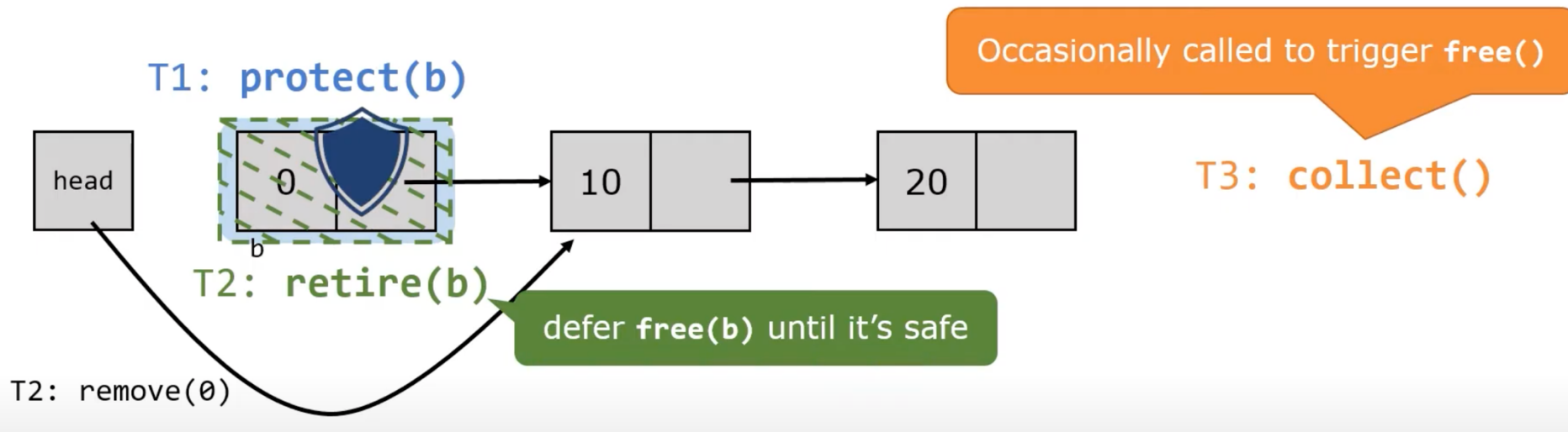


Safe memory reclamation in lock-free world

- Hazard Pointer
- Epoch-based reclamation
- RCU (you can use it beyond memory reclamation)

Hazard Pointer API

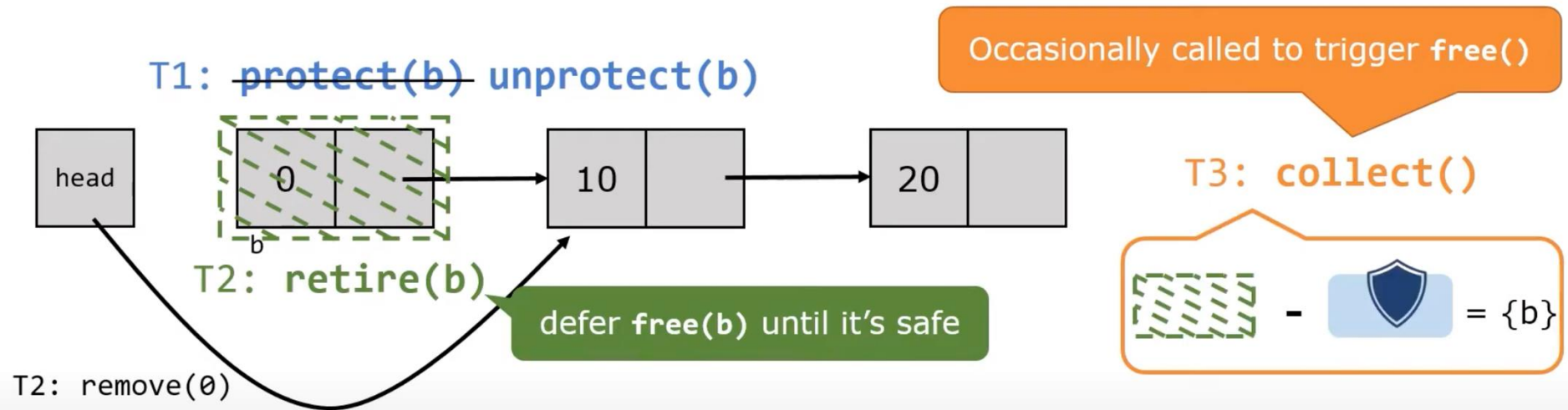
- `lock(b)/unlock(b) → protect(b)/unprotect(b)`
- `free(b) → retire(b)`
- Background thread {
 `list safe-to-free = collect()`
 foreach in the list: `free`



Hazard Pointer Internal

T1's protecting	T2's protecting
b	

- Collect() = Retired - Protected



Hazard Pointer vs Epoch-based reclamation (EBR)

- Hazard Pointer
 - Read(key x)
 - For **each** node n in list
 - Protect(n); //To ensure no others free it in between
 - ...
 - Unprotect(n);
 - 2 memory writes for each n
- EBR: batch them, and free the whole batch

```
Read(key x)
  ebr-enter();
  for each node n in list {
    ...
  }
  ebr-leave();
```

```
Delete(key x)
  ...
  retire (n);
```

Background thread: for each epoch: collect and free

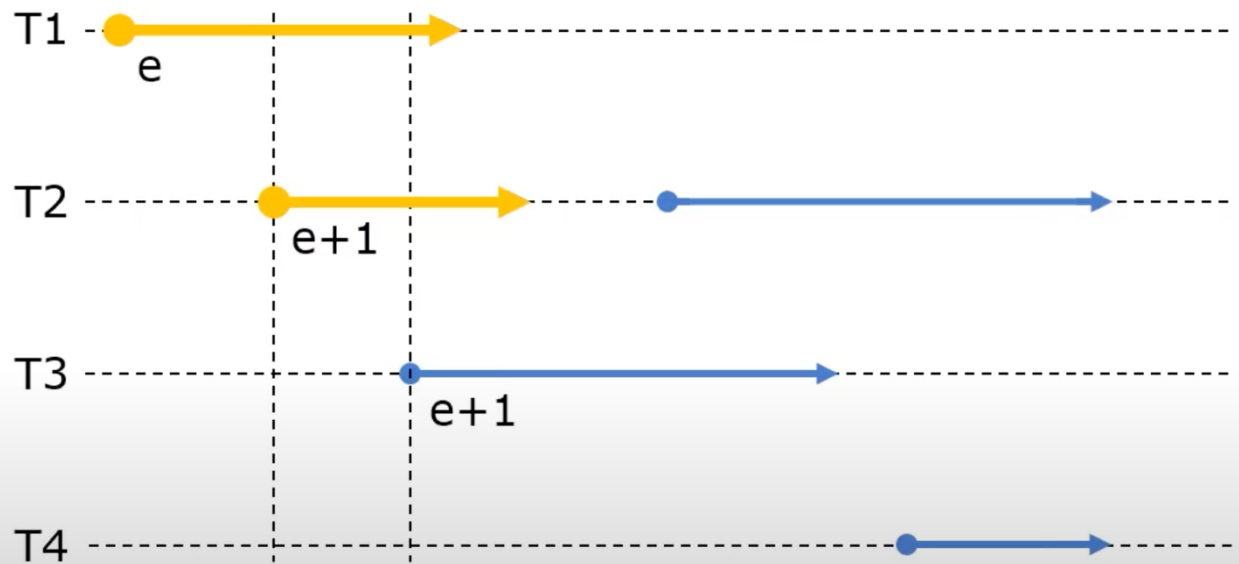
Epoch-based reclamation (EBR) internal

Global Epoch=6	T1	T2	T3	T4
Local Epoch:	5	6	idle	6

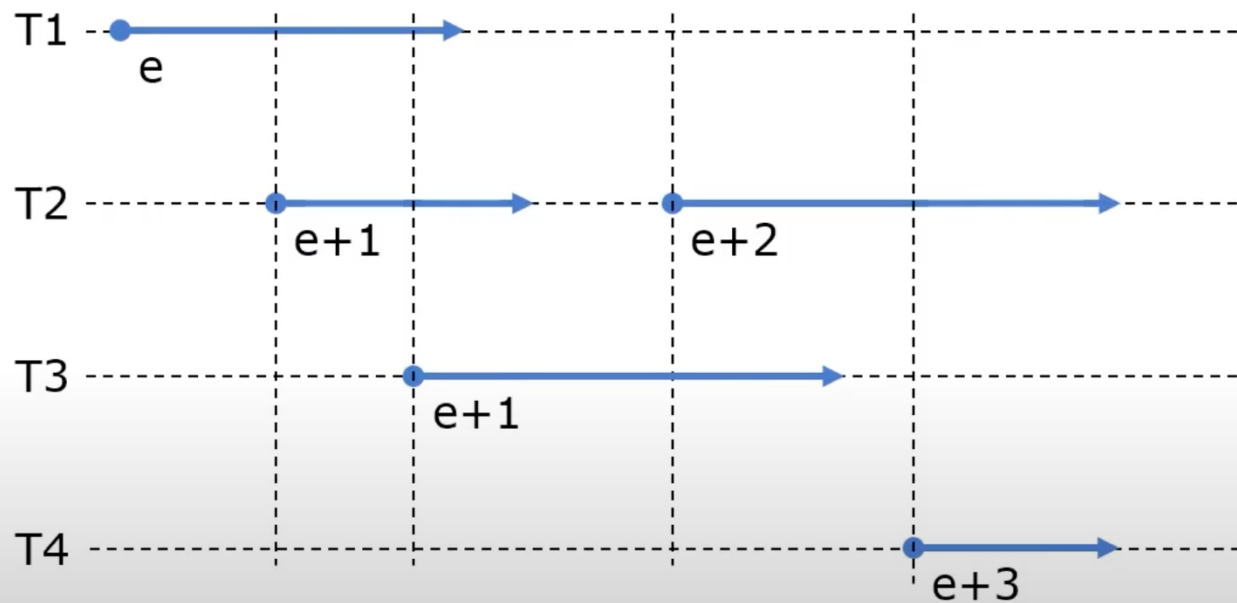
- Rules:
 - Concurrent epochs can differ at most by 1
 - A thread will set its local epoch = global epoch on start
 - A thread will global&local epoch++ when all others local epoch == global epoch
- Retire(n): insert n into epoch e's limbo list
 - E.g., epoch 5 limbo list: {n65, n34}
 - E.g., epoch 6 limbo list: {n71, n18}
- When epoch reaches e (e.g., 7),
 - ➔ it means no more active threads in epoch e-2
 - ➔ free nodes in epoch 5

Epoch consensus rule

Concurrent epochs may differ by at most 1

**Epoch consensus rule**

Concurrent epochs may differ by at most 1



RCU

- A synchronization technique for 1-writer-N-readers
 - Readers: lock-free way to enter a critical section
 - Writer: use a synchronization barrier to separate:
 - Concurrent readers that start
 - before the update (but are still active) could read the **previous version** even if the update has already been installed
 - after the update seeing the **latest version**
- More use inside the kernel as when it leverages the kernel facilities, it is even more efficient
 - Prominent use is still memory-reclamation for kernel's data structure
 - RCU offers lock-free read while giving you a handle to know it is now safe to reclaim the deleted memory

RCU and memory reclamation

- You can implement the concept of RCU yourself
 - But for user-space data-structure, we use EBR more
- But kernels and some user-space library offer you RCU API
- Note: RCU resolves for 1-writer-N-readers
- for Writer-writer conflicts,
 - You add mutex to synchronize between writers
 - When only 1 writer can be active at any time, RCU can apply to that 1 active writer with other readers

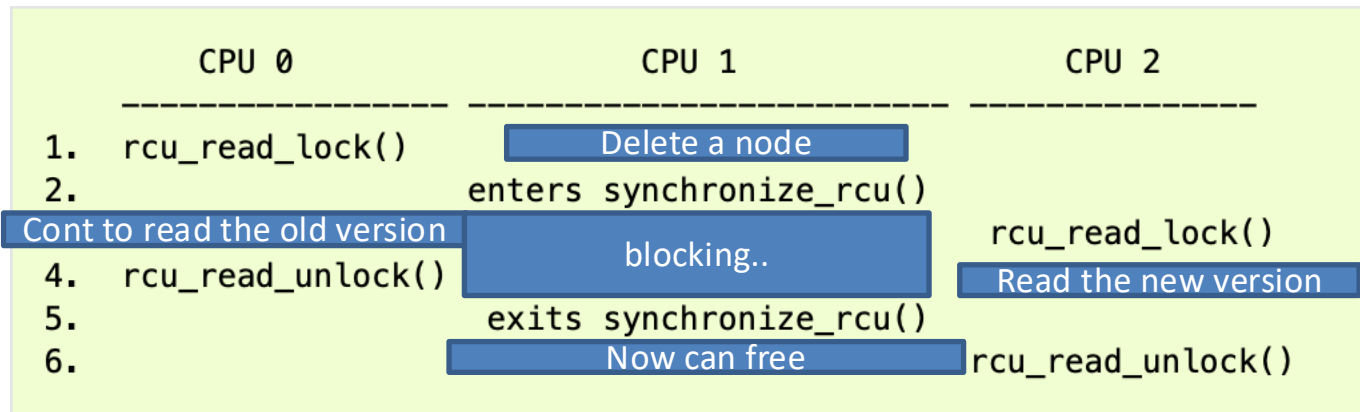
RCU API

- An RCU implementation must ensure no blocking happen within an RCU read-side critical section
 - [rcu_read_lock\(\)](#) //for a reader to "lock" the node before reading it
 - Wait-free Return (i.e., no blocking)
 - Internal implementation:
 - it **writes** no lock, hence no shared memory write (performance), no deadlock to bother you for sure
 - [rcu_read_unlock\(\)](#)
 - Reader call this when finish reading the node
- [synchronize_rcu\(\)](#) / [call_rcu\(\)](#)
- [rcu_assign_pointer\(\)](#)
- [rcu_dereference\(\)](#)

RCU API

- [synchronize_rcu\(\)](#)
 - Call this when a writer finishes
 - E.g., delete(X) { //a writer deletes a key X
node = node.remove (X);
if node.size =0 { //empty node
 [synchronize_rcu\(\)](#); //block until all readers who
read the old version have finished
 free(node); //now it is safe
}

RCU



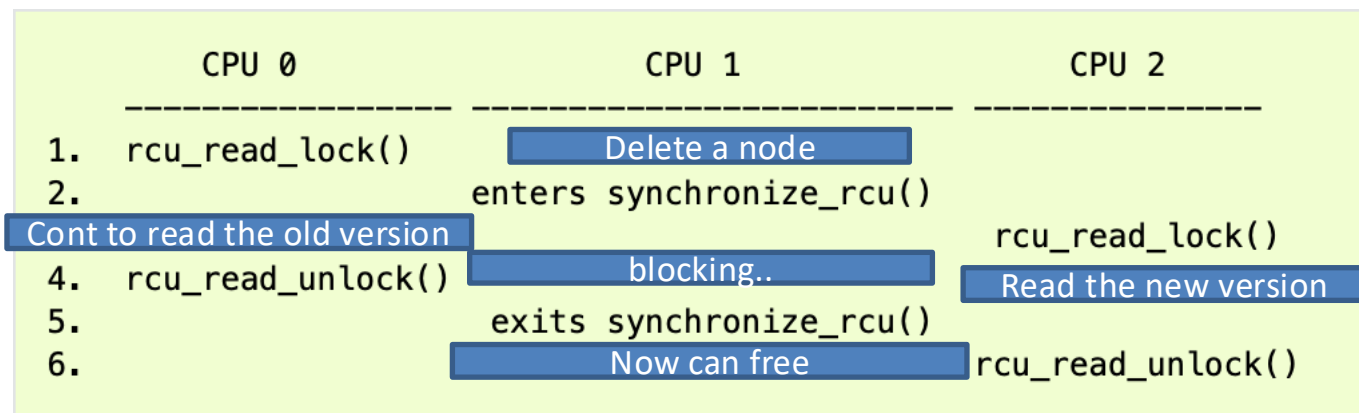
RCU

- [call rcu\(\)](#): the callback version of `synchronize_rcu`
- How can you know the ptr is getting the old version and new version? You don't need to worry:

- [rcu_assign_pointer\(\)](#)

- [rcu_dereference\(\)](#)

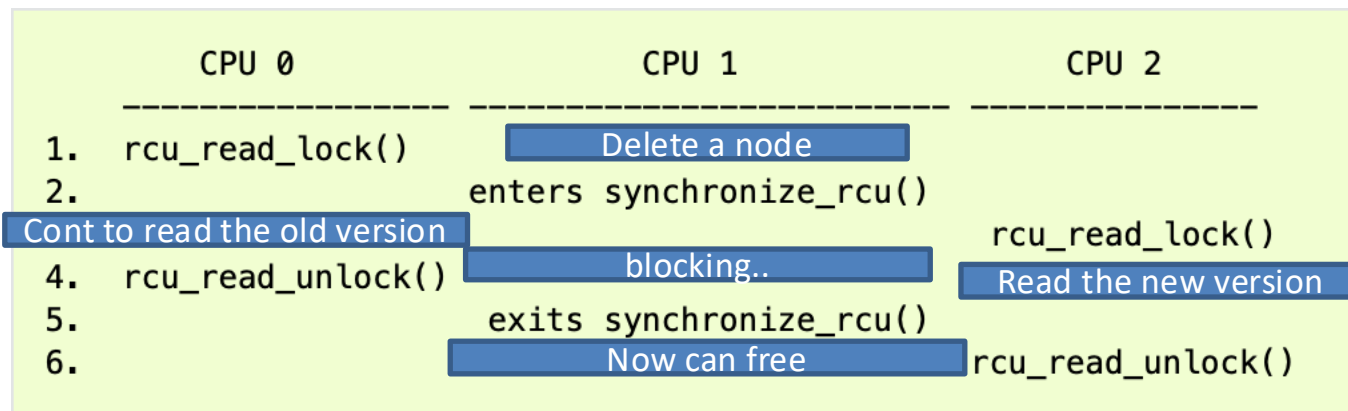
```
ptr = rcu_dereference(head.next);  
return ptr->data;
```



Toy RCU Implementation (assume non-preempt kernel)

- `synchronize_rcu()` makes sure the writer thread runs on each core once
- Since the kernel confines that *there is no context switch when one in an RCU read-side critical section*, hence
 - If a core executes a context switch, we know that it must have completed a RCU read-side critical section
 - Once **each** core has executed a context switch, then **all** RCU read-side critical sections starts before `synchronize_rcu()` would have completed.
 - Then can return

```
void rcu_read_lock(void) { }  
  
void rcu_read_unlock(void) { }  
  
void synchronize_rcu(void)  
{  
    int cpu;  
  
    for_each_possible_cpu(cpu)  
        run_on(cpu);  
}
```



More

- C++20 Co-routine and Software Pre-fetching
 - Latency hiding – remember overlapping compute and communication?
 - Co-routine: lightweight compute and memory access overlapping

Summary: programming angle

- Making it correct is already hard enough
- Performance concerns make it even harder
 - Lock too little? Wrong
 - Lock too much? Can't scale
 - High-performance Lock-free is basically implemented by superhuman